

Exercise 4 for MA-INF 2213 SS26

Handout: 17.06.2026 Submission: 08.07.2026

Generative Neural Networks

Task 1: Image Inpainting

Image inpainting is the task of reconstructing missing or corrupted regions of an image in a way that is visually plausible and coherent with the surrounding context. Unlike unconditional generation, an inpainting model must be conditioned on the visible pixels and reason about what content is consistent with the local and global context of the masked region.

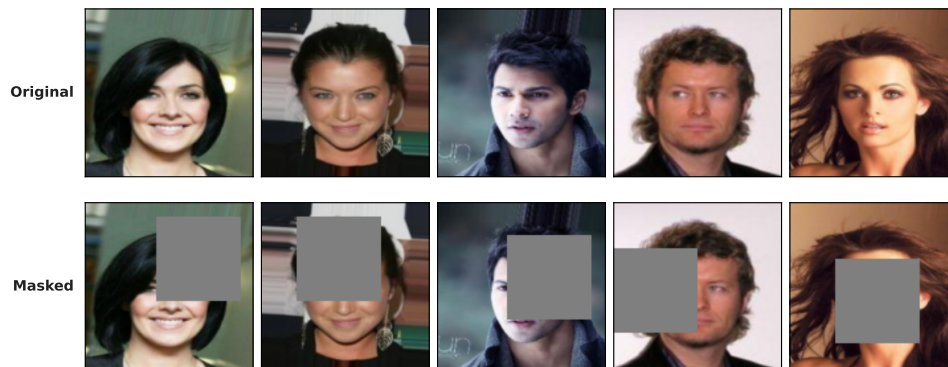


Figure 1: Sample images from the CelebA dataset [1]. Your task is to develop a model that can inpaint masks that cover 10-30% of the original image.

In this task you will train a generative model to perform inpainting on the CelebA dataset [1]. You are free to use any type of generative model that you want (e.g., diffusion models, GANs, variational auto-encoders, ...). During training, random masks are applied to each image. The model receives the masked image as input and must predict the complete, unmasked image. A data download as well as sample script (*data_loading.py*) showing how to load the dataset is provided on sciebo. **For runtime efficiency please resize images to 64×64 .**

Please consider the following for your solution:

1. Implement a dataloader with on-the-fly mask generation. Masks should cover between 10-30% of the original image. The mask should be returned as a separate tensor so that metrics can be computed on the masked region only.
2. Implement at least one generative model (e.g. GAN, diffusion model) of your choice for the inpainting task. Describe and justify your architecture in the report.
3. Train your model with a suitable loss function. You may combine multiple loss components. Discuss the role of each loss component in your report.
4. Discuss sample quantitative and qualitative results. Provide a set of representative success and failure cases with explanations, and a reflection on how architectural and loss design choices influence the observed inpainting performance.

(10 points)

Task 2: Conditional Image Generation

Conditional image generation is the task of synthesizing images that reflect one or more specified attributes. Unlike unconditional generation, the model must learn to disentangle and control semantic traits in the output, producing samples that are both visually realistic and faithful to the given conditioning signal.

In this task you will train a generative model to perform attribute-conditional image generation, again, on the CelebA dataset [1]. You are again free to use any type of generative model (e.g., conditional GAN, conditional diffusion model, conditional VAE, ...) or modify your model(s) from Task 1. You are also free to choose which attributes from the CelebA label set you condition on and how you evaluate the quality of conditioning. Same as Task 1, you use the data download provided on scieb. **For runtime efficiency please resize images to 64×64 .**

Please consider the following for your solution:

1. Select a meaningful subset of CelebA attributes to condition on. Justify your choice of attributes in the report, considering aspects such as visual distinguishability, class balance, and potential interactions between traits.
2. Implement a dataloader that provides images together with the selected attribute labels as conditioning signals.
3. Implement at least one conditional generative model of your choice. Describe and justify your architecture and conditioning mechanism (e.g., label embedding, classifier-free guidance, conditional batch normalisation) in the report.
4. Train your model with a suitable loss function. You may combine multiple loss components. Discuss the role of each component and how they encourage both realism and attribute faithfulness.
5. Discuss sample quantitative and qualitative results. Provide a set of representative success and failure cases with explanations, and reflect on how your design choices affect the trade-off between image quality and attribute control.

(10 points)

Notes. Please exclude the data folders from your submission due to their size. For grading, in addition to evaluating the report and code correctness, we also value the cleanliness and organization of the code.

Questions. If you have questions or find any ambiguities/ mistakes, please contact: Marius Bock (bock@iai.uni-bonn.de) or Sassan Mokhtar (mokhtar@iai.uni-bonn.de).

Important: Please stick to Python 3.12.9, Numpy 2.3.5, and OpenCv 4.12.0.

Grading: Submissions that generate runtime errors or produce obviously useless results (e.g. nans, inf or meaningless visual outputs) will receive at most 50% of the points.

Plagiarism: Plagiarism is prohibited. If your solution contains copied codes (i.e., from other students), you will receive 0 points for the entire exercise sheet.

Submission: You can complete the exercise in a group of two, but only one submission per group is allowed. Include a README.txt file with your group members into each solution. Points for solutions without a README file will only be given to the uploader.

References

- [1] LIU, Z., LUO, P., WANG, X., AND TANG, X. Deep Learning Face Attributes in the Wild. In *2015 IEEE International Conference on Computer Vision (ICCV)* (Santiago, Chile, 2015), IEEE, pp. 3730–3738.